C05｜应用怎样安全读到预约，并留下能追踪的线索

本章把关系接回应用：**客户参数 → 固定查询 → 当前结果行 → Python 字典列表 → 调用者使用**；旁边用一次处理的标识、结果与耗时记录运行线索。它采用独立读取库，客户和技术员编号直接在预约表上。

哪个库、哪些行、谁交回值，是三件什么事?

当前问题	关系或短用法	记到什么程度
查不到数据，先核对文件还是客户编号?	数据库路径决定打开哪份库；客户参数决定在该库中选哪些行。	**访问目标与查询条件分开。** 改客户编号不能补回缺失的表或 view。
看板为什么恰好是 A-11、A-13?	JOIN 接上客户和技术员，筛 C-1 且排除 cancelled，再按时间和预约 ID 排序。	能沿真实主键匹配、条件与顺序解释两行；每行仍是一条预约，组合结果没有写回底表。
engine、connection、execute 各做什么?	`create_engine(...)` 建立连接管理入口，通常延后实际连接；`with engine.connect()` 取得本次连接；`connection.execute(...)` 执行。	认熟创建入口、取得连接、执行查询三个动作；URL 配置不等于查询已发生。
`text()`、Result、字典列表能混为一谈吗?	`text(SQL)` 包装可执行语句；execute 返回 Result；`mappings().all()` 按列名取得全部行； `dict(row)` 转成普通字典。	**语句 → 执行结果 → 应用值**的方向要清楚。text 不读库，修改返回字典不会自动改表。
with 结束，是不是每次都关掉全部数据库连接?	离开 connect 块释放本次使用并结束未提交事务；底层连接可能归池复用。整个实验收尾用 `engine.dispose()` 释放当前空闲连接。	能解释用完归还，不用背连接池参数。取完小结果再返回；巨大结果另查分页或分批。

已有 `CUSTOMER_QUERY` 和 `engine` 时，常用交接片段是:

Python 示例

```
with engine.connect() as connection:
    result = connection.execute(CUSTOMER_QUERY, {"customer_id": customer_id})
    rows = [dict(row) for row in result.mappings().all()]
```

`return rows` 再把列表交给调用者。代码不需要先建立 ORM 类；本章使用 SQLAlchemy Core。SQLAlchemy 里的 `:customer_id` 与 sqlite3 小例里的 `?` 都是值的位置，完整写法随实际入口查。

初始化、锁定依赖与 `make_engine` 回调是长配方，查原章相邻完整程序。关键关系是：SQLite 外键要在**每条新底层连接**、事务外启用并读回；初始化时开过不够。本章初始化用 `autocommit=True` 管固定脚本，应用用 `autocommit=False`；不要把两处配置混成同一模式。示例 `c05_setup.py` 仅首次建新文件，反复读取不用重复初始化。

回看：「客户编号怎样变成两行看板数据?」「怎样把这份小数据库准备好，让 Python 真正读一次?」「这段程序里，谁接过参数，谁交回结果?」。

共用读取规则和安全传值，分别解决什么？

当前问题	关系或短用法	记到什么程度
看板与通知程序怎样共用取消规则？	`CREATE VIEW upcoming_appointments AS ...` 保存共同查询定义；消费者再按客户筛选并 ORDER BY。	view 用来稳定复用列与含义；**普通 view 不复制长期结果快照，也不替消费者保证顺序。**
名字叫 upcoming，就自动只查未来吗？	本章定义只排除 cancelled，没有日期条件；需要未来或当天，仍要明确添加时间范围。	读定义承诺，不凭名字补出规则。客户改名后，后续读取也随底表当前值变化。
客户输入带引号，为什么不会变成 SQL？	`WHERE customer_id = :customer_id` 加独立参数字典；`"C-1' OR 1=1 --"` 整段仍作为一个编号值。	逐渐用熟绑定；**SQL 结构与输入值分开交付**，不是先拼接再清洗危险字符。
绑定安全，是否意味着只能看自己的客户？	合法绑定 C-2 仍能读 C-2。应用必须由可信身份确定可读客户，或先核对权限。	**绑定保护结构，授权决定能看谁。** view 的过滤也不自动授予或限制访问权。
表名、列名、ASC / DESC 也能放参数位置吗？	绑定位置承载值；动态排序等结构应由有限选项映射到应用写好的查询。	分清值与语法，不能把用户文字拼进结构，再指望参数绑定补救。

一处直接查询已经够用时，无需为了缩短 SQL 创建永久 view。普通 view 能被有底表权限的代码绕过；SQLite 文件访问权限、应用授权与服务器数据库账号权限，分别在各自位置处理。

回看：「看板和通知程序怎样共用同一种读取规则？」「客户编号里带引号，为什么也只能是一个值？」。

成功空结果与读取失败，怎样留下不同的线索？

当前问题	关系或短用法	记到什么程度
连续读同一客户两次，怎样区分是哪一次？	客户 / 预约 ID 指业务对象；request ID 由可信应用入口产生，传给本次各项工作；下一次处理用新标识。	**业务身份与处理身份分开。** 本例传 UUID 对象；它不防重放，也不证明调用者有权限。
怎样描述一次数据库读取？	固定 operation 加 outcome、row_count、elapsed_ms、error_type，使用同一个 request ID 关联。	能解释字段回答的具体问题；日志是运行线索，不是另一张预约表。
为什么没有记录是 0，读取失败是 null？	成功返回 [] 表示确知零行；执行失败尚未得到答案，数量未知。	**查过但没有，与没能查出分开。** 不能失败后悄悄返回空列表，冒充正常答案。
elapsed_ms 就是纯 SQL 耗时吗？	本例计时从调用读取函数到取完行、结束连接使用；`perf_counter()` 两次差值乘 1000 得毫秒。	先认计时范围再归因；不是预约日期，也不能把全部等待直接归到 SQL 执行。
为什么日志只写异常类别，不直接 str(error)？	原数据库异常可能携带 SQL、参数与底层信息。逐个选择日志字段，向调用者交固定提示。	熟悉最小必要线索；`hide_parameters=True`、`raise ... from None` 只是各自位置的保护，不能替字段选择。

C04 的事件与预约、时段一起提交；本章 print 出的运行日志没有自动加入那个事务。日志写了“成功”，不能补出未提交的业务事实；一次数据库事件也不会自动包含所有运行耗时。

原章通过在合成库暂时移除 view，观察“执行查询时失败”，再恢复定义并读回。这种失败不表示客户不存在，改编号也修不好入口。完整受控演示与异常处理查用即可；成功日志的行数来自已取得的列表，失败日志不用原始客户内容填充。

回看：「查询失败了，怎样把线索关联到那一次处理？」「怎样看到真实失败，又不把异常内容写进日志？」。

汇总、明细编号与稳定入口，为什么不是同一种工具？

当前问题	关系或短用法	记到什么程度
每人预约数，与每条预约的组内编号差在哪？	GROUP BY 折成每组一行；`ROW_NUMBER() OVER (PARTITION BY technician_id ORDER BY starts_at, appointment_id)` 在明细旁编号，保留预约行。	**先决定是否保留明细。** 本例汇总是两行，窗口编号仍是三行。
窗口里写了 ORDER BY，外面还需要吗？	窗口排序决定谁得到哪个编号；最外层 ORDER BY 决定答案怎样展示。	直接分清计算顺序与显示顺序，前者不替代后者。
WITH 的名字能下次继续读吗？	`WITH filtered AS (...)` 给当前语句的中间结果命名；普通 view 则保存供后续读取的定义。	**名字寿命与结果形状是两个维度。** CTE、view 都能与聚合或窗口组合，不承诺自动加速。
从预约汇总，会自动出现零预约技术员吗？	`technician_load` 从未取消预约出发，没有这类预约的人不出现；要零值需从技术员 LEFT JOIN 并数非空预约主键。	先看输入从谁开始。本例 open_count 是“未取消数”，不等于筛一个 open 状态，也非全部历史。
底表改列，怎样保住消费者？	本例新增 display_name 并回填，view 仍以 `AS customer_name` 交出原公开列名；提交前比较列名和完整样本结果。	**内部结构可变，公开列与语义仍要守住。** 只确认新列建成，不足以证明读取兼容。
新列、回填、换 view 中间失败，怎样恢复？	放入 `with engine.begin()`：正常结束提交，异常离开回滚；依赖原章已配置的事务模式。	完整迁移与故障读回按配方查；不能仅看一个 with 就断言任何驱动下 DDL 都可回滚。

本例迁移保留旧姓名列，只验证一次读取变化；新旧两列以后不会自己同步。成功迁移后不能重复执行同一新增列步骤，也不能拿旧 view 脚本覆盖新定义。查看公开列与样本结果是在保护交接，不是要求背下整份迁移。

完整 CTE、报告与迁移程序查原章；SQLAlchemy、sqlite3 的具体接口再从原章官方入口核对目标版本。配套程序还使用不同的日志字段与 request ID 约定，相近名字不意味着可直接互换。

回看：「每位技术员的预约数，与每条预约的组内编号，是同一个结果吗？」「底表要增加姓名列，怎样保住消费者已经依赖的入口？」「忘记具体接口写法时，从哪里查回？」。